Full-Stack Web Application :

Part A – Keywords (Concepts & Technologies)

🚀 Technical Stack

React

ספריית JavaScript לבניית ממשקי משתמש מבוססי רכיבים (**Components**), המאפשרת פיתוח צד-לקוח דינמי ומהיר.

TypeScript

גרסה מתקדמת של JavaScript המוסיפה **Static Typing**, מה שעוזר למנוע שגיאות בזמן הפיתוח ומשפר את הסדר בקוד.

Node.js

סביבת ריצה המאפשרת להריץ קוד JavaScript בצד השרת (**Backend**), ידועה במהירות וביעילות שלה בטיפול בבקשות רבות.

Express.js

מסגרת עבודה (**Framework**) פופולרית ל Node.js-שמפשטת את בניית השרת, ניהול הנתיבים (**Routing**) וה- Middleware.

MongoDB

מסד נתונים מסוג **NoSQL** השומר מידע בצורה גמישה מה שמאפשר שינויים מהירים במבנה הנתונים.

Mongoose

ספרייה (ODM) המקשרת בין Node.js ל - MongoDB, ומאפשרת להגדיר מבנה נתונים ברור (**Schema**) וולידציות בצד השרת.

Material UI

ספריית רכיבי עיצוב מוכנים ל- React, המאפשרת לבנות ממשקים יפים ומקצועיים במהירות לפי תקני העיצוב של גוגל.

Vercel / Render / Railway

פלטפורמות ענן המאפשרות העלאה (**Deployment**) מהירה של אפליקציות ושרתים לאינטרנט ישירות מ- GitHub.

⚙️ Functional & Features

User Registration

תהליך יצירת חשבון חדש הכולל קבלת פרטים, אימותם (**Validation**), הצפנת הסיסמה ושמירה של המשתמש בבסיס הנתונים.

Login System

מערכת המאמתת את זהות המשתמש מול המידע השמור ב DB-ומנפיקה אישור כניסה להמשך הגלישה.

Logout Functionality

ניתוק המשתמש מהמערכת על ידי מחיקת ה Token-מהדפדפן וביטול הרשאות הגישה בשרת.

Account Verification

תהליך אימות החשבון (לרוב דרך מייל) כדי לוודא שהמשתמש אמיתי ולמנוע הרשמות פיקטיביות.

Money Transfer

תהליך העברת כספים המחייב אבטחה מקסימלית ושימוש ב- **Transactions** כדי להבטיח שהפעולה הושלמה במלואה או בוטלה במקרה של תקלה.

Unique Email Validation

בדיקה בבסיס הנתונים שמוודאת כי כתובת המייל שסופקה אינה רשומה כבר תחת משתמש אחר.

Protected Routes

נתיבים באפליקציה שנגישים רק למשתמשים מחוברים או בעלי הרשאות מתאימות, תוך חסימת משתמשים אורחים.

Role-Based Access (RBAC)

ניהול הרשאות לפי תפקידים) למשל Admin :מול ,(User שקובע אילו פעולות כל משתמש רשאי לבצע במערכת.

Customer Service Chat Bot

ממשק שיחה אוטומטי העונה לשאלות משתמשים ומשפר את השירות והזמינות של האתר.

Error Handling

ניהול שגיאות בקוד בצורה חכמה, המונע קריסה של האפליקציה ומציג למשתמש הודעה ברורה במקום שגיאה טכנית.

Security & Architecture

Authentication

תהליך זיהוי המשתמש (מי אתה?), המתבצע לרוב על ידי שם משתמש וסיסמה או דרך צד שלישי כמו גוגל.

Encryption

הצפנת מידע רגיש כך שגם אם הנתונים ייגנבו, לא ניתן יהיה לקרוא אותם ללא המפתח המתאים.

JWT Tokens

אמצעי מאובטח להעברת מידע בין הלקוח לשרת בצורה של מחרוזת חתומה, המשמשת בעיקר לשמירת המשתמש מחובר.

Secure REST API

ממשק תקשורת המוגן על ידי הצפנת **HTTPS**, אימות משתמשים וחסימת גישות לא מורשות.

Session Security

הגנה על החיבור של המשתמש מפני גניבת זהות, למשל על ידי שימוש ב Cookies-מאובטחים או פקיעת תוקף של טוקנים.

API Rate Limiting

הגבלת מספר הבקשות שמשתמש יכול לשלוח לשרת בזמן נתון, כדי למנוע עומס או התקפות מסוג **DoS**.

CORS

מנגנון אבטחה בדפדפן המגדיר אילו אתרים מורשים לגשת למשאבים של השרת שלך ומתי לחסום בקשות זרות.

Database Schema

הגדרה מראש של המבנה והסוג של הנתונים שיישמרו בבסיס הנתונים, מה שמבטיח סדר ועקביות במידע.

🧪 Development & Testing

Unit Testing

בדיקה של יחידת קוד קטנה ובודדת (כמו פונקציה) כדי לוודא שהיא מחזירה את התוצאה הצפויה בכל מצב.

Integration Testing

בדיקה שבוחנת כיצד כמה חלקים במערכת עובדים יחד, למשל החיבור בין השרת לבסיס הנתונים.

End-to-End (E2E) Testing

בדיקה המדמה מסלול מלא של משתמש באפליקציה (מהלוגין ועד הקנייה) כדי לוודא שהכל עובד מקצה לקצה.

Cypress

כלי פופולרי להרצת בדיקות **Frontend** ו- E2E המאפשר לראות את הבדיקה רצה בתוך דפדפן בזמן אמת.

Jest

ספריית בדיקות מהירה ל JavaScript-ו React, המשמשת בעיקר לכתיבת Unit Tests.

Supertest

ספרייה לבדיקת שרתי Node.js המאפשרת לשלוח בקשות HTTP לשרת ולוודא שהתגובות תקינות.

CI/CD Pipeline

תהליך אוטומטי המריץ בדיקות ומעלה את הקוד לשרת בכל פעם שמתבצע שינוי.

Docker Containerization

אריזת האפליקציה וכל רכיביה בתוך "מכולה" (**Container**) שרצה באותה צורה בכל מחשב או שרת.

Swagger / Swagger UI

כלים ליצירת תיעוד ויזואלי ואינטראקטיבי של ה- API, המאפשרים למפתחים להבין ולהתנסות בבקשות השרת בקלות.

Postman

אפליקציה לבדיקה וניהול של בקשות API, המאפשרת לשלוח בקשות HTTP ולראות את התגובה מהשרת.

Twilio API

שירות חיצוני המאפשר לשלוח הודעות SMS, לבצע שיחות טלפון או וואטסאפ מהקוד שלך.

Socket.IO

ספרייה המאפשרת תקשורת דו-כיוונית בזמן אמת (**Real-time**) בין השרת למשתמשים.

Jitsi

תשתית קוד פתוח לניהול שיחות וידאו שניתן להטמיע ישירות בתוך אתר אינטרנט.

Nodemailer

ספרייה לשליחת מיילים אוטומטיים משרת (Node.js) כמו מייל ברוכים הבאים.

Figma

כלי עיצוב (UI/UX) המשמש לתכנון המראה הוויזואלי של האפליקציה לפני שמתחילים לכתוב את הקוד.

Part B – Interview Questions & Answers

🚀 Technical Stack

Q: What is React and how does it help build interactive UIs?

ספריית JavaScript לבניית ממשקי משתמש (UIs) מבוססי **Components**. היא משתמשת ב-**Virtual DOM** כדי לעדכן רק את החלקים שהשתנו בדף, מה שיוצר חוויית גלישה מהירה ואינטראקטיבית ללא ריענון הדף כולו.

Q: How does TypeScript improve reliability and maintainability in large projects?

שפה שמוסיפה **Static Typing** ל- JavaScript, היא עוזרת למצוא באגים בזמן הפיתוח (**Compile time**) ומשפרת את ה- **Maintainability** בפרויקטים גדולים בזכות הגדרת סוגי נתונים ברורים לכל משתנה ופונקציה.

Q: How does Express.js simplify server-side routing and middleware?

ספריית Backend מינימליסטית ל- Node.js. היא מפשטת **Routing** (ניתוב בקשות) וניהול **Middleware** (פונקציות ביניים) לטיפול בבקשות HTTP, מה שמאפשר בניית שרתים בקלות ובמהירות.

Q: What is MongoDB and how is data stored differently than in SQL databases?

מסד נתונים מסוג **NoSQL** השומר מידע בפורמט דמוי JSON הנקרא **BSON**. בניגוד ל SQL-המבוסס טבלאות קשיחות, כאן המידע גמיש (**Schema-less**), מה שמאפשר שינויים מהירים במבנה הנתונים.

Q: What problem does JWT authentication solve, and how does it work?

פתרון לאימות משתמשים ללא שמירת מצב בשרת (**Stateless**). השרת מנפיק **Token** חתום המכיל את פרטי המשתמש. הדפדפן שולח אותו בכל בקשה, והשרת מוודא את תקינותו ללא צורך בגישה למסד הנתונים בכל פעם.

Q: What is a REST API and what are its core principles?

ארכיטקטורה לתקשורת בין מערכות המבוססת על פרוטוקול **HTTP**. העקרונות כוללים שימוש ב-**Methods** (כמו GET, POST), עבודה ללא שמירת מצב (**Statelessness**) והפרדה ברורה בין ה-Client ל- Server.

Q: How does Swagger help document and test APIs?

כלי לתיעוד (**Documentation**) ובדיקה של APIs. הוא מייצר ממשק ויזואלי המאפשר למפתחים לראות אילו **Endpoints** קיימים, אילו פרמטרים הם דורשים, ולהריץ בדיקות ישירות מהדפדפן.

Q: What is Postman used for in backend development?

כלי עבודה למפתחי Backend המאפשר לשלוח בקשות HTTP לשרת, לבדוק תגובות, לנהל סביבות עבודה (**Environments**) ולכתוב טסטים אוטומטיים ל- API עוד לפני שיש ממשק משתמש מוכן.

Q: How does Docker help in deploying consistent environments?

כלי המאפשר להריץ אפליקציות בתוך **Containers** מבודדים הכוללים את כל הקוד
וה- Dependencies הנדרשים. זה מבטיח שהאפליקציה תרוץ באותה צורה בדיוק על המחשב של המפתח ועל השרת (**Consistent environments**).

Q: Why use Material UI in a React project?

ספריית רכיבים מוכנה (**UI Library**) ל React-המבוססת על עקרונות העיצוב של גוגל. היא מאפשרת לבנות ממשקים יפים, נגישים ו- **Responsive** במהירות מבלי לכתוב CSS מורכב מאפס.

Q: What services like Vercel, Render, or Railway offer for app hosting?

שירותי ענן (**Cloud Hosting**) המפשטים את תהליך ה- **Deployment**. הם מתחברים ל- GitHub ומעלים את האתר או השרת לאוויר באופן אוטומטי בכל פעם שמעדכנים את הקוד.

Q: What is Cypress and how is it used for frontend testing?

כלי לבדיקות **End-to-End (E2E)** של ה .Frontend-הוא מריץ דפדפן אמיתי ומדמה פעולות משתמש (כמו לחיצה על כפתורים ומילוי טפסים) כדי לוודא שהאפליקציה עובדת מקצה לקצה.

Q: What is Jest and what types of tests can it perform?

ספריית **Testing** רב-תכליתית המשמשת בעיקר ל- **Unit Tests** (בדיקת פונקציה בודדת)
ו- **Integration Tests**. היא מהירה מאוד ומגיעה עם כלי מובנה לבדיקת כיסוי הקוד.

Q: What is Supertest and how is it used in backend API testing?

ספרייה לבדיקת API ב- Node.js. היא מאפשרת לשלוח בקשות HTTP מדומות לשרת ה- Express ולוודא שהתגובות (**Status codes, Data**) תקינות, ללא צורך להריץ את השרת ידנית.

Q: What is Socket.IO and how does it enable real-time communication?

ספרייה המאפשרת תקשורת דו-כיוונית בזמן אמת (**Real-time**) בין הלקוח לשרת. היא משתמשת ב- **WebSockets** ומאפשרת לבנות יישומים כמו Chat או התראות קופצות באופן מיידי.

Q: How is Jitsi used for implementing video calls in a web app?

תשתית קוד פתוח המאפשרת להטמיע שיחות וידאו (**Video calls**) באפליקציות WEB. ניתן להשתמש ב API-שלהם כדי להוסיף חדר ישיבות וירטואלי ישירות בתוך האתר ללא פיתוח מורכב של פרוטוקולי וידאו.

Q: What is Nodemailer and how do you send emails from Node.js?

ספריית Node.js לשליחת אימיילים. היא מאפשרת להתחבר לשרתי **SMTP** (כמו Gmail) ולשלוח הודעות טקסט או HTML מהשרת באופן אוטומטי (למשל באיפוס סיסמה).

Q: Why would you use Figma in a full-stack project workflow?

כלי לעיצוב ממשקים (**UI/UX Design**). ב Full-stack, משתמשים בו כדי לתכנן את המראה והזרימה של האתר לפני הקידוד, מה שחוסך זמן עבודה יקר ומבטיח תיאום בין העיצוב למימוש.

⚙️ Functional / Feature

Q: How does user registration typically work in web applications?

תהליך בו המשתמש מזין פרטים בטופס, ה Client-שולח אותם ל Server, שם מתבצעת ולידציה, הצפנת סיסמה (**Hashing**), ושמירה של ה User-במסד הנתונים.

Q: What are the steps in a secure login system?

מערכת הכוללת זיהוי המשתמש, אימות הסיסמה המוצפנת מול ה DB, והנפקת **Token** (כמו JWT) לזיהוי עתידי. מומלץ להוסיף מנגנונים כמו **Rate Limiting** למניעת התקפות Brute Force.

Q: What should happen when a user logs out?

פעולה שבה ה- Client מוחק את ה- **Token** (מה- Local Storage או ה- Cookies) בצד השרת, אם משתמשים ב- **Refresh Tokens**, יש לבטל (**Invalidate**) אותם במסד הנתונים כדי למנוע שימוש חוזר.

Q: Why is account verification important and how can it be implemented?

תהליך אישור זהות, לרוב דרך אימייל עם קוד או לינק חד-פעמי. זה קריטי למניעת בוטים, הבטחת נכונות המידע, ויכולת לשחזר את החשבון בעתיד.

Q: How does a money transfer process ensure data accuracy and security?

תהליך המחייב שימוש ב- **Database Transactions** כדי להבטיח שפעולת החיוב והזיכוי יקרו יחד (Atomic). נדרשת ולידציה קפדנית של יתרה ואבטחה ברמת ה API-למניעת זיופים.

Q: How do you ensure that each email is unique in user registration?

כדי להבטיח אימייל ייחודי, מגדירים **Unique Constraint** במסד הנתונים על השדה. בשרת, לפני השמירה, מבצעים שאילתה שבודקת אם המייל כבר קיים ומחזירים שגיאה מתאימה למשתמש.

Q: What is session management and how does JWT help with it?

ניהול מצב המשתמש לאורך זמן JWT. עוזר בזה בכך שהשרת לא צריך לזכור מי מחובר (**Stateless**). המידע "ארוז" בתוך ה Token-שהמשתמש מחזיק.

Q: How are JWT tokens created, validated, and refreshed?

השרת יוצר Token באמצעות **Secret Key** (חתימה). בבדיקה, השרת מפענח את החתימה. אם היא תואמת, המידע בפנים אמין. **Refresh Tokens** משמשים לחידוש ה- Session ללא צורך בהתחברות מחדש.

Q: How should errors be handled gracefully in a web application?

טיפול בשגיאות בצורה אלגנטית על ידי שימוש ב- **Try-Catch** ו- **Middleware** מרכזי.למשתמש מחזירים הודעה ידידותית וקוד סטטוס HTTP מתאים (כמו 404), בעוד שבשרת שומרים לוג מפורט.

Q: How can a customer service chatbot improve user experience?

כלי המבוסס על תסריטים קבועים או AI הנותן מענה מיידי לשאלות נפוצות. הוא משפר את חוויית המשתמש על ידי זמינות 24/7 והורדת עומס מנציגי שירות אנושיים.

Q: How Password kept in DB?

לעולם לא שומרים סיסמה כטקסט פשוט. משתמשים בספרייה כמו **Bcrypt** כדי לבצע **Hashing** עם "Salt", מה שהופך את הסיסמה לרצף תווים בלתי ניתן לפענוח גם אם ה DB-נפרץ.

Security & Architecture

Q: What is authentication and how is it implemented in Node.js apps?

תהליך אימות זהות המשתמש. מיושם לרוב בעזרת ספריות כמו **Passport.js** או על ידי מימוש ידני של JWT. הכולל השוואת סיסמאות מוצפנות ויצירת Token גישה.

Q: What is encryption and where is it used in a banking system?

הצפנת נתונים כך שרק גורם מורשה יוכל לקרוא אותם. במערכת בנקאית משתמשים ב- **SSL/TLS** להצפנה בזמן מעבר (**In transit**) ובאלגוריתמים כמו **AES** להצפנת המידע בתוך ה- DB.

Q: What role do JWT tokens play in securing communication?

ה Token-משמש כ"תעודת זהות" חתומה דיגיטלית. הוא מאפשר לשרת לדעת מי המשתמש ומה ההרשאות שלו בכל בקשה, מבלי לשלוח סיסמה שוב ושוב, מה שמקטין את הסיכון ליירוט מידע רגיש.

Q: How can you secure a REST API against unauthorized access?

הגנה הכוללת שימוש ב- **HTTPS**, אימות (Auth), הגבלת כמות בקשות (**Rate Limiting**), ולידציה של הקלט למניעת הזרקות קוד, ושימוש ב -**CORS** כדי להגביל מי יכול לפנות לשרת.

Q: Why is data validation important, and what libraries help achieve it?

בדיקה שהקלט מהמשתמש תקין ומאובטח. ספריות כמו **Joi** או **Yup** עוזרות להגדיר חוקים ברורים ולחסום מידע שגוי או זדוני לפני שהוא מגיע לשרת.

Q: How does session security protect user data?

הגנה על הקשר בין המשתמש לשרת. זה כולל שימוש ב- **HttpOnly Cookies** (למניעת גניבת טוקנים ע"י סקריפטים), הגדרת זמן תפוגה קצר לטוקנים, ושימוש ב- **Secure flags**.

Q: What is CORS, and why must it be configured properly in web apps?

מנגנון אבטחה בדפדפן המונע מאתר אחד לבקש מידע משרת של אתר אחר ללא אישור מפורש. הגדרה נכונה (**Allowed Origins**) מבטיחה שרק האתרים שלכם יוכלו להשתמש ב API-שלכם.